



Guía de Estudio — Programación Móvil (6NM61)

Examen: 8 de mayo 2026

Enfoque del examen: Cómo funciona cada función, su propósito, y qué funcionalidad estándar aporta cada API.

Índice

1. UI: News — Card y Composables	3
1.1. Funciones clave	3
1.2. Composable con parámetros	3
1.3. Versión mejorada: <code>buildAnnotatedString</code>	3
2. UI: News Improved — LazyColumn y Device ID	3
2.1. Funciones clave	3
2.2. LazyColumn vs Column con scroll	4
2.3. Device ID	4
3. UI: Jukebox — LazyVerticalGrid y MediaPlayer	4
3.1. Funciones clave	4
3.2. Modelo de datos	4
3.3. Flujo de reproducción	5
4. Sensors: Acelerómetro y SensorManager	5
4.1. API de Sensores — Arquitectura	5
4.2. Funciones clave	5
4.3. SensorEventListener	5
4.4. DisposableEffect — Registrar y limpiar	6
4.5. Canvas — Dibujo personalizado	6
4.6. BoxWithConstraints	6
5. Data: DataStore Preferences	6
5.1. Funciones clave	7
5.2. Flujos	7
6. Data: SQLite con SQLiteOpenHelper	7
6.1. SQLiteOpenHelper	7
6.2. CRUD completo	7
6.3. Tabla de funciones SQLite	8
6.4. Modelo: <code>data class Product</code>	8
7. Transversales: State, Intent, Ciclo de Vida	9
7.1. <code>remember</code> y <code>mutableStateOf</code>	9
7.2. Intent — Navegación entre Activities	9
7.3. Ciclo de Vida + Refresh	9
7.4. LaunchedEffect	9
8. Preguntas Tipo Examen	10
8.1. UI	10
8.2. Sensors	10
8.3. DataStore	10
8.4. SQLite	10

9. Cheat Sheet 12

1. UI: News — Card y Composables

Propósito: Aprender a crear componentes de UI reutilizables con parámetros.

1.1. Funciones clave

Función/API	¿Qué hace?
<code>@Composable fun News(...)</code>	Función composible que recibe datos como parámetros y dibuja una tarjeta de noticia
<code>Card(modifier, shape, elevation)</code>	Contenedor Material 3 con sombra y bordes redondeados
<code>RoundedCornerShape(4.dp)</code>	Define esquinas redondeadas de 4dp para la Card
<code>CardDefaults.cardElevation(4.dp)</code>	Configura la sombra/elevación de la Card
<code>Column { }</code>	Apila hijos verticalmente (uno debajo del otro)
<code>Row { }</code>	Apila hijos horizontalmente (uno al lado del otro)
<code>Modifier.fillMaxWidth()</code>	El elemento ocupa todo el ancho disponible
<code>Modifier.background(Color.Black)</code>	Pinta el fondo del contenedor de negro
<code>Modifier.padding(16.dp)</code>	Agrega espacio interno de 16dp

1.2. Composable con parámetros

```
@Composable
fun News(title: String, author: String, date: String, content: String) { ... }
```

Los composables se llaman como funciones normales pasando datos. **No usan XML** — la UI se construye 100% en Kotlin.

1.3. Versión mejorada: `buildAnnotatedString`

En `news-improved`, el composable usa texto con estilos mezclados:

```
Text(text = buildAnnotatedString {
    append("Publicado por:\n")
    withStyle(SpanStyle(fontWeight = FontWeight.Bold, fontStyle = FontStyle.Italic)) {
        append(author)
    }
})
```

Función	¿Qué hace?
<code>buildAnnotatedString { }</code>	Construye un string con diferentes estilos por sección
<code>append(text)</code>	Agrega texto con estilo normal
<code>withStyle(SpanStyle(...)) { }</code>	Agrega texto con estilo específico (negrita, cursiva, etc.)

2. UI: News Improved — LazyColumn y Device ID

Propósito: Mostrar listas largas eficientemente y leer identificadores del dispositivo.

2.1. Funciones clave

Función/API	¿Qué hace?
<code>LazyColumn { items(list) { } }</code>	Lista scrolleable que solo renderiza los elementos visibles en pantalla

Función/API	¿Qué hace?
<code>items(newsList) { (t,a,d) -> }</code>	Itera sobre la lista; el destructuring extrae los valores del <code>Triple</code>
<code>Settings.Secure.getString(...)</code>	Lee el ID único del dispositivo Android (persistente entre reinicios)
<code>LocalContext.current</code>	Obtiene el <code>Context</code> de Android desde dentro de un Composable
<code>@Preview</code>	Anotación que permite ver el composable en el editor sin ejecutar la app

2.2. LazyColumn vs Column con scroll

<code>Column</code> + <code>verticalScroll</code>	<code>LazyColumn</code>
Renderiza todos los elementos de golpe	Renderiza solo los visibles
OK para listas cortas (<20 items)	Necesario para listas largas
Usa <code>Modifier.verticalScroll(...)</code>	Usa DSL <code>items(list) { }</code>

2.3. Device ID

```
val deviceId = Settings.Secure.getString(
    context.contentResolver,
    Settings.Secure.ANDROID_ID
)
```

- `contentResolver` es el intermediario para acceder a datos del sistema
- `ANDROID_ID` es un identificador hexadecimal de 64 bits, único por dispositivo + usuario

3. UI: Jukebox — LazyVerticalGrid y MediaPlayer

Propósito: Crear una cuadrícula interactiva que reproduce sonidos al tocar imágenes.

3.1. Funciones clave

Función/API	¿Qué hace?
<code>LazyVerticalGrid(columns = GridCells.Fixed(3))</code>	Cuadrícula scrolleable con 3 columnas fijas
<code>GridCells.Fixed(n)</code>	Número fijo de columnas
<code>GridCells.Adaptive(minSize)</code>	Columnas adaptativas según el ancho disponible
<code>painterResource(id = R.drawable.xxx)</code>	Carga una imagen desde <code>res/drawable/</code>
<code>Modifier.clickable { }</code>	Hace que cualquier composable responda a toques
<code>MediaPlayer.create(context, R.raw.xxx)</code>	Crea un reproductor de audio desde <code>res/raw/</code>
<code>mediaPlayer.start()</code>	Inicia la reproducción
<code>setOnCompletionListener { it.release() }</code>	Al terminar de sonar, libera la memoria
<code>Modifier.size(100.dp)</code>	Fija el tamaño del elemento a 100×100dp

3.2. Modelo de datos

```
data class JukeboxItem(val label: String, val imageRes: Int, val soundRes: Int)
```

Agrupar etiqueta + imagen + sonido en un solo objeto. Los `Int` son IDs de recurso (`R.drawable.*`, `R.raw.*`).

3.3. Flujo de reproducción

- 1. Usuario toca imagen → se dispara `clickable { }`
- 2. `MediaPlayer.create(context, soundRes)` → crea instancia lista para sonar
- 3. `setOnCompletionListener { it.release() }` → al terminar, libera memoria
- 4. `start()` → reproduce el audio

Limitación: Si tocas rápido varias veces, se crean múltiples instancias que suenan simultáneamente (audio superpuesto).

4. Sensors: Acelerómetro y SensorManager

Propósito: Leer sensores de hardware en tiempo real y usarlos para controlar la UI.

4.1. API de Sensores — Arquitectura



4.2. Funciones clave

Función/API	¿Qué hace?
<code>getSystemService(SENSOR_SERVICE)</code> as <code>SensorManager</code>	Obtiene el servicio del sistema que administra todos los sensores
<code>sensorManager.getDefaultSensor(TYPE_ACCELEROMETER)</code>	Obtiene el acelerómetro. Retorna <code>null</code> si no hay uno
<code>registerListener(listener, sensor, delay)</code>	Registra un listener para recibir eventos del sensor
<code>unregisterListener(listener)</code>	Des-registra el listener — imprescindible para evitar fugas
<code>SensorManager.SENSOR_DELAY_GAME</code>	Frecuencia rápida (~20ms), ideal para juegos
<code>SensorManager.SENSOR_DELAY_UI</code>	Frecuencia más lenta (~60ms), suficiente para UI

4.3. SensorEventListener

```

object : SensorEventListener {
    override fun onSensorChanged(event: SensorEvent) {
        x = event.values[0] // Inclinación lateral
        y = event.values[1] // Inclinación adelante/atrás
    }
    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
}

```

Método	¿Cuándo se llama?
<code>onSensorChanged(event)</code>	Cada vez que el sensor reporta nuevos valores (según el delay)
<code>onAccuracyChanged(...)</code>	Cuando cambia la precisión del sensor (raramente usado)

4.4. DisposableEffect — Registrar y limpiar

```
DisposableEffect(Unit) {  
    sensorManager.registerListener(listener, acelerometro, SENSOR_DELAY_GAME)  
    onDispose {  
        sensorManager.unregisterListener(listener)  
    }  
}
```

Parte	¿Qué hace?
<code>DisposableEffect(Unit)</code>	Ejecuta código al entrar en composición, con bloque de limpieza
Bloque principal	Registra el listener (se ejecuta una vez)
<code>onDispose { }</code>	Se ejecuta cuando el composable sale de pantalla
<code>Unit</code> como key	El efecto solo se ejecuta una vez (key nunca cambia)

Importante: Si no llamas `unregisterListener` en `onDispose`, el sensor **sigue enviando datos** después de salir de la pantalla, desperdiciando batería y memoria.

4.5. Canvas — Dibujo personalizado

```
Canvas(modifier = Modifier.fillMaxSize()) {  
    drawRect(color = Color.Black, topLeft = Offset(...), size = Size(...))  
    drawCircle(color = Color.Red, radius = radius, center = Offset(posX, posY))  
}
```

Función	¿Qué hace?
<code>Canvas { }</code>	Composable que permite dibujo libre 2D
<code>drawRect(color, topLeft, size)</code>	Dibuja un rectángulo
<code>drawCircle(color, radius, center)</code>	Dibuja un círculo
<code>Offset(x, y)</code>	Punto 2D para posicionar elementos
<code>Size(width, height)</code>	Dimensiones de un rectángulo

4.6. BoxWithConstraints

```
BoxWithConstraints(modifier = Modifier.weight(1f)) {  
    val width = constraints.maxWidth.toFloat()  
    val height = constraints.maxHeight.toFloat()  
}
```

Igual que `Box` pero expone `constraints.maxWidth/maxHeight` para saber el tamaño disponible en píxeles.

5. Data: DataStore Preferences

Propósito: Almacenar preferencias de usuario de forma asíncrona y segura.

5.1. Funciones clave

Función/API	¿Qué hace?
<code>preferencesDataStore("nombre")</code>	Crea una instancia singleton de DataStore
<code>stringPreferencesKey("clave")</code>	Crea una clave tipada que solo acepta <code>String</code>
<code>context.dataStore.data</code>	Retorna un <code>Flow<Preferences></code> — stream reactivo
<code>.map { prefs -> prefs[clave] ?: "" }</code>	Transforma el Flow, extrae el valor con default
<code>dataStore.edit { prefs -> prefs[clave] = v }</code>	Escritura atómica. Es <code>suspend</code>
<code>.collectAsState(initial = "")</code>	Convierte <code>Flow</code> → <code>State</code> de Compose
<code>rememberCoroutineScope()</code>	Scope para lanzar coroutines desde un composable
<code>scope.launch { }</code>	Ejecuta una coroutine sin bloquear la UI

5.2. Flujos

Lectura:

```
dataStore.data → Flow<Preferences> → .map {} → Flow<String> → .collectAsState() → State → UI
```

Escritura:

```
Button onClick → scope.launch { dataStore.edit { prefs[clave] = valor } }
```

`edit` es `suspend` → necesita coroutine → `scope.launch { }` la ejecuta sin bloquear.

6. Data: SQLite con SQLiteOpenHelper

Propósito: Base de datos relacional local para CRUD completo.

6.1. SQLiteOpenHelper

Método	¿Cuándo se llama?	¿Qué hace?
<code>onCreate(db)</code>	Solo la 1ª vez (.db no existe)	<code>CREATE TABLE</code>
<code>onUpgrade(db, old, new)</code>	Cuando subes <code>DATABASE_VERSION</code>	Migración (DROP + CREATE)

Constructor: `SQLiteOpenHelper(context, "shop.db", null, 2)`

- `context` → dónde guardar el archivo
- `"shop.db"` → nombre del archivo de BD
- `null` → cursor factory (casi siempre null)
- `2` → versión del esquema

6.2. CRUD completo

6.2.1. INSERT

```
writableDatabase.compileStatement("INSERT INTO products (...) VALUES (?, ?, ?, ?)")
    .use { stmt ->
        stmt.bindString(1, name)    // Los ? son 1-indexed
        stmt.bindDouble(2, price)
        stmt.bindLong(4, stock)
```

```

        stmt.executeInsert() // → retorna el ID del nuevo registro
    }

```

6.2.2. SELECT (todos)

```

readableDatabase.rawQuery("SELECT * FROM products", null).use { cursor ->
    while (cursor.moveToNext()) {
        cursor.getString(cursor.getColumnIndexOrThrow("name"))
    }
}

```

6.2.3. SELECT (por ID)

```

readableDatabase.rawQuery("SELECT * FROM products WHERE id = ?", arrayOf(id.toString()))
    .use { cursor ->
        if (cursor.moveToFirst()) { /* encontrado */ }
    }

```

6.2.4. UPDATE

```

writableDatabase.compileStatement("UPDATE products SET name=? WHERE id=?")
    .use { stmt -> stmt.bindString(1, name); stmt.bindLong(2, id); stmt.executeUpdateDelete() }

```

6.2.5. DELETE

```

writableDatabase.compileStatement("DELETE FROM products WHERE id=?")
    .use { stmt -> stmt.bindLong(1, id); stmt.executeUpdateDelete() }

```

6.3. Tabla de funciones SQLite

Función	Propósito
<code>writableDatabase</code>	Abre BD en modo escritura
<code>readableDatabase</code>	Abre BD en modo lectura (más eficiente)
<code>compileStatement(sql)</code>	Pre-compila SQL con marcadores <code>?</code> (previene SQL injection)
<code>rawQuery(sql, args)</code>	Ejecuta SELECT, retorna <code>Cursor</code>
<code>execSQL(sql)</code>	Ejecuta DDL sin retorno (CREATE, DROP)
<code>.use { }</code>	Cierra automáticamente el recurso al terminar
<code>bindString/Double/Long(i, val)</code>	Enlaza valor al marcador <code>?</code> en posición <code>i</code>
<code>executeInsert()</code>	Ejecuta INSERT, retorna row ID
<code>executeUpdateDelete()</code>	Ejecuta UPDATE/DELETE, retorna # filas afectadas
<code>cursor.moveToNext()</code>	Avanza al siguiente registro; <code>false</code> = no hay más
<code>cursor.moveToFirst()</code>	Va al primer registro; <code>false</code> = resultado vacío
<code>getColumnIndexOrThrow(col)</code>	Índice de columna por nombre (excepción si no existe)

6.4. Modelo: `data class Product`

```

data class Product(val id: Long = 0, val name: String, val price: Double,
    val country: String, val stock: Int)

```


`data class` auto-genera: `equals()`, `hashCode()`, `toString()`, `copy()`, destructuring.
`copy()` se usa en edit: `product.copy(name = "nuevo")` — crea copia con campos modificados.

7. Transversales: State, Intent, Ciclo de Vida

7.1. `remember` y `mutableStateOf`

```
val products by remember(refreshKey) { mutableStateOf(db.getAll()) }
```

Pieza	Propósito
<code>mutableStateOf(value)</code>	Contenedor observable — cambios provocan recomposición
<code>remember { }</code>	Memoriza el valor entre recomposiciones
<code>remember(key) { }</code>	Re-ejecuta el bloque si <code>key</code> cambia
<code>by</code>	Delegación — usa <code>products</code> directamente sin <code>.value</code>

7.2. Intent — Navegación entre Activities

Operación	Código
Navegar	<code>startActivity(Intent(context, Target::class.java))</code>
Enviar dato	<code>.putExtra("product_id", id)</code>
Recibir dato	<code>intent.getLongExtra("product_id", -1L)</code>
Regresar	<code>activity.finish()</code>

7.3. Ciclo de Vida + Refresh

```
class MainActivity : ComponentActivity() {  
    private val refreshKey = mutableStateOf(0)  
    override fun onResume() { super.onResume(); refreshKey.value++ }  
}
```

`onResume` se llama cada vez que la Activity vuelve al frente → incrementa key → `remember(key)` re-ejecuta → datos frescos.

7.4. `LaunchedEffect`

```
LaunchedEffect(Unit) { activity.finish() }
```

Ejecuta un efecto secundario **una sola vez** al entrar en composición. Con `Unit` como key, no se re-ejecuta.

8. Preguntas Tipo Examen

8.1. UI

P: ¿Cuál es la diferencia entre `LazyColumn` y `LazyVerticalGrid` ?

R: `LazyColumn` muestra items en una sola columna vertical. `LazyVerticalGrid` los organiza en una cuadrícula de N columnas. Ambos solo renderizan los elementos visibles.

P: ¿Qué hace `painterResource(id)` ?

R: Carga una imagen desde `res/drawable/` y la convierte en un `Painter` que `Image` puede mostrar.

P: ¿Por qué se llama `release()` en el `MediaPlayer` ?

R: `MediaPlayer` consume recursos nativos. `release()` los libera. Sin ella, cada toque acumularía recursos hasta agotar la memoria.

P: ¿Qué hace `Modifier.clickable { }` ?

R: Hace que cualquier composable detecte toques del usuario y ejecute el bloque lambda.

8.2. Sensors

P: ¿Qué retorna `getDefaultSensor(Sensor.TYPE_ACCELEROMETER)` ?

R: Un objeto `Sensor`, o `null` si el dispositivo no tiene acelerómetro. Siempre hay que verificar `null`.

P: ¿Para qué sirve `DisposableEffect` con sensores?

R: Para registrar el listener al entrar en composición y des-registrarlo al salir (`onDispose`). Sin esto, el sensor sigue enviando eventos desperdiciando batería.

P: ¿Qué contiene `event.values` del acelerómetro?

R: Un array de 3 floats: `values[0]` = eje X (lateral), `values[1]` = eje Y (adelante/atrás), `values[2]` = eje Z (gravedad).

P: ¿Diferencia entre `SENSOR_DELAY_GAME` y `SENSOR_DELAY_UI` ?

R: `GAME` actualiza ~cada 20ms (50/seg), ideal para juegos. `UI` actualiza ~cada 60ms (~16/seg), suficiente para pantalla.

8.3. DataStore

P: ¿Qué ventaja tiene `DataStore` sobre `SharedPreferences`?

R: Es completamente asíncrono (coroutines/Flow), no bloquea el hilo principal, y ofrece seguridad de tipos con claves tipadas.

P: ¿Por qué `saveGuardar` es `suspend` ?

R: La escritura a disco es I/O que no debe bloquear el hilo principal. `suspend` obliga a llamarla desde una coroutine.

P: ¿Qué hace `collectAsState(initial = "")` ?

R: Convierte un `Flow` en un `State` de Compose. `initial` es el valor antes de la primera emisión. Cada emisión recompone la UI.

8.4. SQLite

P: ¿Cuándo se ejecuta `onCreate` de `SQLiteOpenHelper` ?

R: Solo la primera vez que la app accede a la BD y el archivo `.db` no existe.

P: ¿Qué pasa si incrementas `DATABASE_VERSION` ?

R: Se llama `onUpgrade`. En este código, borra la tabla y la recrea (migración destructiva).

P: ¿Diferencia entre `readableDatabase` y `writableDatabase` ?

R: `readable` = modo lectura (eficiente). `writable` = lectura + escritura.

P: ¿Por qué usar `?` en vez de concatenar valores al SQL?

R: Los marcadores `?` previenen SQL injection. Nunca concatenes input del usuario al SQL.

P: Explica `val products by remember(refreshKey) { mutableStateOf(db.getAll()) }`

R: `mutableStateOf` crea estado observable. `remember(refreshKey)` memoriza pero re-ejecuta si `refreshKey` cambia. `by` permite acceder sin `.value`. Cuando `onResume` incrementa `refreshKey`, se re-consulta la BD.

P: ¿Qué retorna `executeInsert()`? ¿Y `executeUpdateDelete()`?

R: `executeInsert()` → el **row ID** del nuevo registro. `executeUpdateDelete()` → el **número de filas afectadas**.

9. Cheat Sheet

UI COMPOSABLES

Card(shape, elevation) → Tarjeta M3
Column { } → Vertical
Row { } → Horizontal
LazyColumn { items() } → Lista eficiente
LazyVerticalGrid(cols) → Grid eficiente
Canvas { drawRect/Circle } → Dibujo 2D
BoxWithConstraints → dimensiones
Image(painterResource) → Imagen local
Modifier.clickable { } → Touch handler
buildAnnotatedString → Texto estilizado
@Preview → Vista previa

MEDIA

MediaPlayer.create(ctx, R.raw.x) → crear
.start() → play
.setOnCompletionListener { release() }

SENSORS

getSystemService(SENSOR_SERVICE) → Manager
getDefaultSensor(TYPE_ACCEL) → Sensor?
registerListener(l, s, delay) → escuchar
unregisterListener(l) → parar
event.values[0,1,2] → x, y, z
SENSOR_DELAY_GAME / _UI → velocidad
DisposableEffect + onDispose → limpieza

DATASTORE

preferencesDataStore(<name>) → instancia
stringPreferencesKey(<key>) → clave
.data.map { it[key] ?: <> } → leer Flow
.edit { it[key] = val } → escribir
.collectAsState(initial) → Flow→State
scope.launch { } → coroutine

SQLITE

SQLiteOpenHelper(ctx, name, null, ver)
onCreate(db) → CREATE TABLE (1ª vez)
onUpgrade(db, o, n) → migración
readableDatabase → SELECT
writableDatabase → INSERT/UPDATE/DELETE
compileStatement → SQL con ?
rawQuery(sql, args) → Cursor
executeInsert() → retorna row ID
executeUpdateDelete() → retorna # filas
cursor.moveToNext/First → navegar filas
.use { } → auto-close

COMPOSE STATE

mutableStateOf(v) → observable
remember { } → memorizar
remember(key) { } → re-calc si keyΔ
by → sin .value
LaunchedEffect(key) → efecto 1 vez
DisposableEffect(key) → con limpieza
LocalContext.current → Context

INTENTS

Intent(ctx, Target::class.java) → crear
.putExtra(<key>, value) → enviar
intent.getLongExtra(<key>, -1) → recibir
activity.finish() → regresar

Errores comunes en examen:

- `readableDatabase` para `SELECT`, `writableDatabase` para escritura — no al revés
- `remember` sin key **nunca** re-ejecuta el bloque
- `onCreate` de `SQLiteOpenHelper` se llama **solo la 1ª vez**, no cada vez que abre la app
- `executeInsert()` retorna ID, `executeUpdateDelete()` retorna # filas — no confundir
- `DataStore.edit {}` es `suspend` — necesita `scope.launch { }`
- `DisposableEffect` sin `onDispose` = fuga de recursos con sensores
- `MediaPlayer` sin `release()` = fuga de memoria
- `getDefaultSensor()` puede retornar `null` — siempre verificar